CS683 PA2 – Summary

Task 1: Stride Pattern Prefetcher (L1D); Task 2A: Instruction-conscious L2 replacement (I-RRIP); Task 3: SPP + prefetch-aware Dynamic I-RRIP

Task 1A: IP-stride (baseline prefetcher)

`ip_stride.l1d_pref` keeps 24 IP trackers (IP, last line address, last stride, LRU). On an access by a tracked IP it computes the stride from the last line; if it equals the stored stride it prefetches the next 3 strides ahead (same page only), otherwise it just stores the new stride. It therefore captures only a *constant* stride and is blind to repeating non-uniform sequences such as $(1, 2, 3)^n$. It is tracked by a fully-associative LRU table of 24 entries, so IPs beyond 24 thrash it.

Task 1B: SPP design

- **IP table** (256 entries, direct mapped, 8-bit tag): last line (low 16 bits), last stride, a 2-bit uniform-stride confidence, and a signature of the last 3 strides (7 bits each).
- **Pattern table** (256 entries) indexed by a hash of the 3-stride signature → (next stride, 2-bit confidence). A **fallback table** (64 entries) is keyed by the last 2 strides and is used when the 3-stride entry is not confident.
- **Prediction**: if the uniform confidence is high the constant stride is used; otherwise the pattern table is walked, feeding each predicted stride back into the signature, up to the current degree. Prefetching stops at the 4KB page boundary and when the prefetch queue is full. Only loads/RFOs train. Prefetches fill L1D unless the MSHR is more than half full, then L2.
- **Accuracy throttling**: a 64-entry filter of recently filled prefetch lines; a demand hit on it counts as useful. Every 256 prefetch fills accuracy is computed and the degree set: ≥75%→6, ≥50%→4, ≥25%→2, else 1 (initial 4).

Results (25M warmup + 25M simulated instructions)

Speedup = IPC / baseline IPC (mean = geometric). Coverage = (baseline L1D load misses − misses with prefetcher)/baseline. Accuracy is the simulator's (useful / prefetches issued to the lower level). **IPC**

Config	Trace1	Trace2	Trace3	Mean
No prefetcher	0.2323	0.2512	0.3821	0.2885
IP-stride	0.2335	0.2513	0.3829	0.2892
SPP (ours)	0.2343	0.2549	0.3872	0.2921

Speedup

Config	Trace1	Trace2	Trace3	Mean
No prefetcher	1.0000	1.0000	1.0000	1.0000
IP-stride	1.0048	1.0004	1.0021	1.0024
SPP (ours)	1.0082	1.0149	1.0135	1.0122

L1D load MPKI

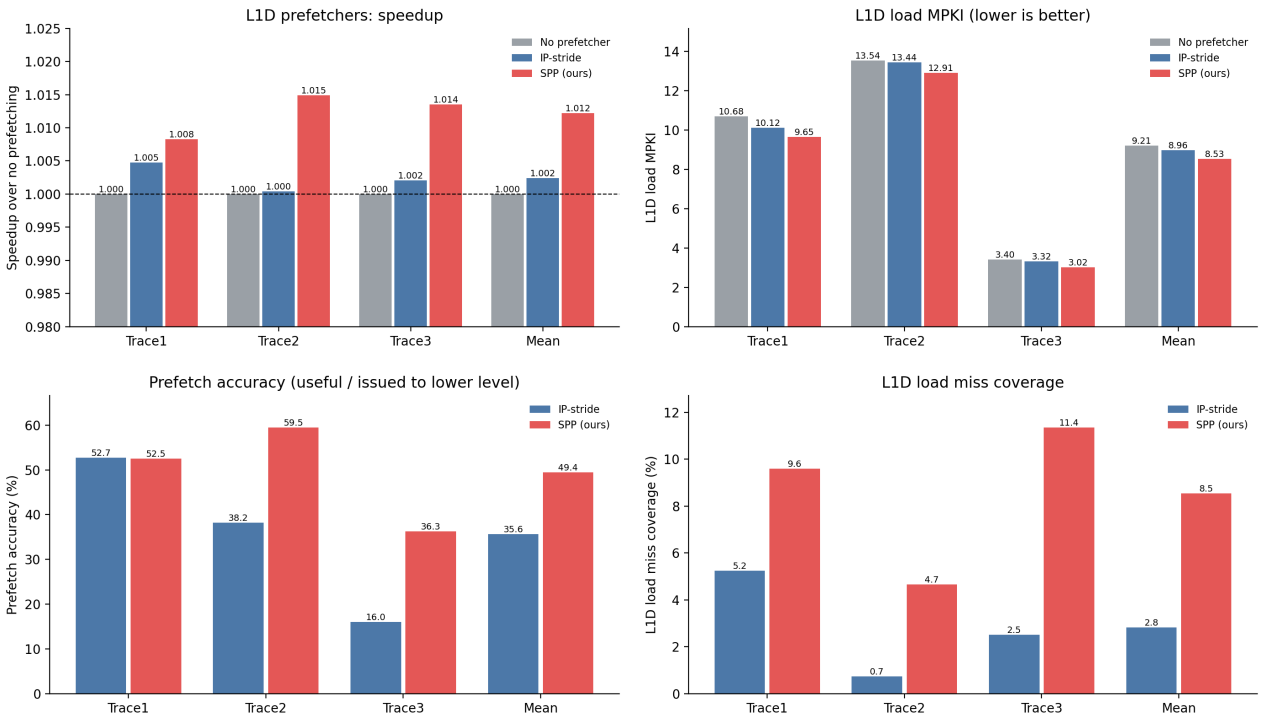
Config	Trace1	Trace2	Trace3	Mean
No prefetcher	10.678	13.538	3.403	9.206
IP-stride	10.119	13.438	3.318	8.958
SPP (ours)	9.653	12.907	3.017	8.526

Prefetch accuracy (%)

Config	Trace1	Trace2	Trace3	Mean
No prefetcher	–	–	–	–
IP-stride	52.71	38.18	15.99	35.63
SPP (ours)	52.49	59.53	36.27	49.43

L1D load coverage (%)

Config	Trace1	Trace2	Trace3	Mean
No prefetcher	–	–	–	–
IP-stride	5.24	0.74	2.51	2.83
SPP (ours)	9.60	4.66	11.36	8.54



SPP beats IP-stride on every trace in speedup and coverage (mean speedup 1.0122 vs 1.0024; coverage 8.5% vs 2.8%), and has higher accuracy on Trace 2 and 3 (tied on Trace 1). Gains are small in absolute terms: a miss breakdown showed most L1D misses come from IPs not in the table, strides beyond a page (forbidden by the spec) or irregular accesses; only about 3–5% of misses occur while a stride pattern is active.

Storage overhead (exact)

Structure	Entries	Bits per entry
IP table	256	valid 1 + tag 8 + last line 16 + last stride 7 + uni conf 2 + 3×7 signature + hist count 2 =
Pattern table	256	stride 7 + conf 2 = 9
Fallback table	64	stride 7 + conf 2 = 9
Recent-prefetch filter	64	valid 1 + tag 10 = 11
Counters	–	issued 9 + useful 9 + degree 3
Total		

For comparison, the provided IP-stride uses 24 × (64+64+64+32) = 5376 bits as coded; SPP uses about 3.4× more storage.

Threshold and degree rationale

Accuracy thresholds 75/50/25% divide accuracy into four bands: above 75% prefetches are almost always used, so aggressive depth (6) is cheap; 50–75% is the normal band (4); 25–50% means over half the prefetches are wasted, so depth is halved (2); below 25% prefetching costs more bandwidth and L1D pollution than it saves, so only one line is issued. The values are round quartile boundaries; I validated them by sweeping: thresholds 50/30/10, 60/40/20, 90/70/40 and degree sets (8,4,2,1), (8,6,4,2) all lie within ±0.1% of mean speedup (1.0115–1.0129), so the result is insensitive to them. Fixed degrees 1/2/4/6 give mean speedups 1.0076/1.0091/1.0101/1.0102, so deeper is better up to 4, and flat beyond. **Honest finding:** dynamic throttling gives the same speedup as fixed degree 4 (1.0122 vs 1.0124) but 4–9 points higher accuracy, i.e. it saves bandwidth without losing performance.

Ablations and design validation

Configuration	Mean speedup	Note
Uniform stride only (no pattern table)	1.0043	≈ IP-stride-like
Pattern table, no fallback	1.0100	
Pattern table + fallback (final)	1.0122	

IP-table size (without fallback): 64/128/256/512/1024 entries → 1.0054/1.0078/1.0100/1.0107/1.0111 (6.7/10.3/17.6/32.2/61.4 Kbit); 256 chosen as the knee. A single 2-stride table alone reached 1.0126, within noise of the final hybrid; the hybrid was kept because the 3-stride signature disambiguates longer patterns. Synthetic tests (uniform, non-uniform, phase changes, random, interleaved IPs) give about 97% coverage on regular streams, about 0% on random, and 0 page-boundary violations in 186K prefetches; the one weakness is interleaved IPs sharing a signature in the pattern table (78%), a known trade-off since IP bits in the index lowered real-trace speedup.

Task 2A: Instruction-conscious L2 replacement (I-RRIP)

All Task 2 runs use no L1D prefetcher, so the replacement policy is evaluated in isolation (Task 3 combines it with SPP). The baseline is LRU at L2.

Exposing instruction information. `update_replacement_state` was extended with a 9th parameter `isInstruction` for every cache (declarations in `inc/cache.h`, all `replacement/*_repl` definitions). The six call sites in `src/cache.cc` pass the packet's `instruction` field (MSHR entry on fills, RQ/WQ/PQ entry on hits); the BTB calls in `src/ooo_cpu.cc` pass 0. For eviction, the policy reads the per-line `BLOCK::instruction` bit, which `fill_cache` already sets for L2C lines filled by instruction loads/prefetches, through the `current_set` argument of `find_victim`.

Policy (`replacement/irrip.l2c_repl`, 2-bit RRPV as in SRRIP):

- **Insertion:** instruction lines at RRPV 1, data lines at RRPV 3 (SRRIP: both at 2).
- **Promotion:** an instruction hit sets RRPV to 0; a data hit decrements RRPV by 1 (SRRIP: both to 0).
- **Eviction:** among lines at RRPV 3, a data line is evicted before an instruction line; aging is unchanged.

Results (25M warmup + 25M simulation instructions; speedup is geometric mean over the three traces). **IPC**

Config	Trace1	Trace2	Trace3	Mean
Baseline (LRU)	0.2322	0.2513	0.3819	0.2885
SRRIP	0.2322	0.2518	0.3816	0.2885
I-RRIP	0.2319	0.2597	0.3826	0.2914

Speedup over baseline

Config	Trace1	Trace2	Trace3	Mean
Baseline (LRU)	1.0000	1.0000	1.0000	1.0000
SRRIP	0.9999	1.0022	0.9993	1.0004
I-RRIP	0.9986	1.0334	1.0020	1.0112

L2C load MPKI

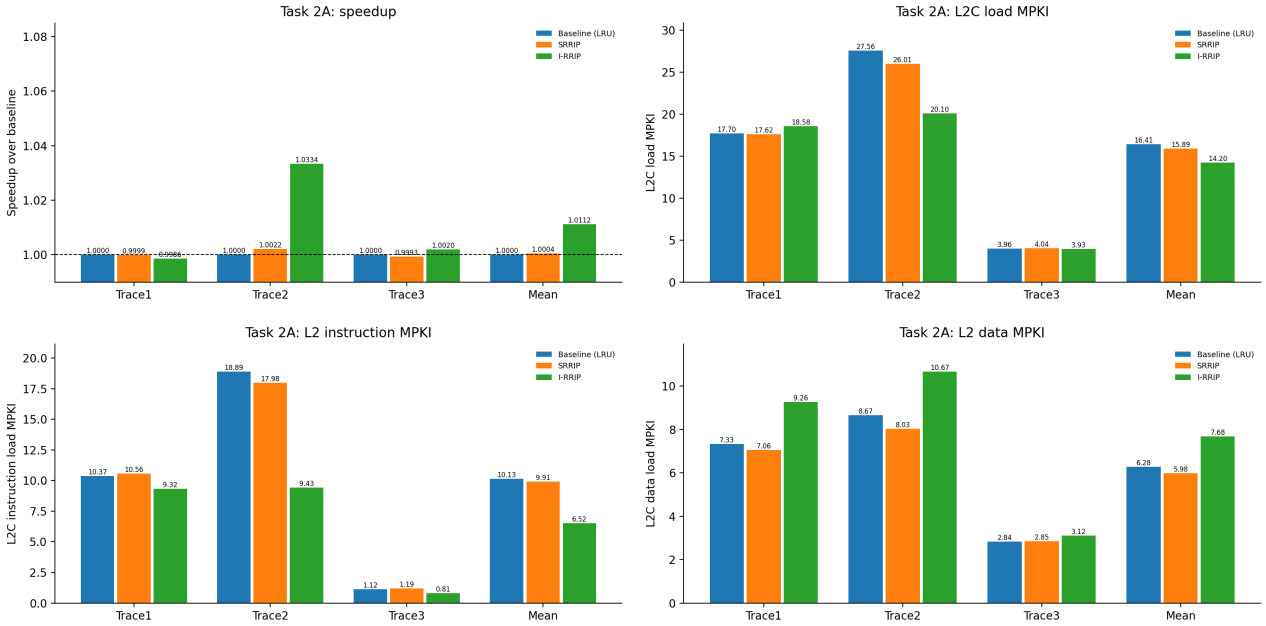
Config	Trace1	Trace2	Trace3	Mean
Baseline (LRU)	17.703	27.560	3.960	16.408
SRRIP	17.619	26.012	4.037	15.889
I-RRIP	18.578	20.104	3.928	14.204

L2C instruction load MPKI

Config	Trace1	Trace2	Trace3	Mean
Baseline (LRU)	10.368	18.892	1.124	10.128
SRRIP	10.563	17.980	1.190	9.911
I-RRIP	9.316	9.435	0.811	6.520

L2C data load MPKI

Config	Trace1	Trace2	Trace3	Mean
Baseline (LRU)	7.334	8.668	2.837	6.280
SRRIP	7.056	8.032	2.846	5.978
I-RRIP	9.263	10.669	3.118	7.683



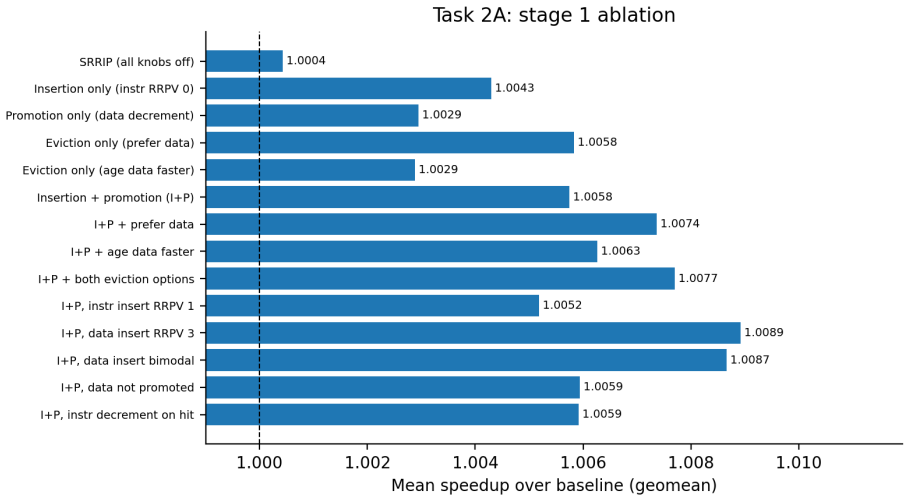
Findings. I-RRIP reaches a mean speedup of 1.0112, against 1.0004 for SRRIP. It cuts the mean L2 instruction

load MPKI from 9.91 (SRRIP) to 6.52 (−34%), at the cost of data load MPKI rising from 5.98 to 7.68 (+28%); total L2 load MPKI still falls 11% (15.89 to 14.20), because instruction misses are both the larger share and the more costly ones (an instruction miss stalls the front end, while out-of-order execution hides part of a data miss). The gain is concentrated on Trace 2, the trace with by far the highest L2 instruction MPKI: instruction MPKI almost halves (17.98 to 9.44) and IPC improves 3.3%. Trace 3 gains 0.2%. Trace 1 loses 0.14%: its LLC misses about 69% of the loads it receives under every policy, so the extra L2 data misses go to DRAM, and they outnumber the instruction misses saved (data MPKI +2.21 versus instruction MPKI −1.25 relative to SRRIP). Blindly prioritising instructions is therefore not always right, which is the motivation for Task 2B.

How the design was chosen. Every part of the policy was made a compile-time knob (`replacement/irrip_policies.l2c_repl`): instruction insertion RRPV (0/1/2), data insertion (RRPV 2 / RRPV 3 / bimodal: RRPV 3 with every 32nd insertion at 2), instruction promotion (to 0 / decrement), data promotion (to 0 / decrement / unchanged), and two eviction options (prefer a data victim among RRPV-3 lines; age data lines by 2 instead of 1). The full grid is $3 \cdot 3 \cdot 2 \cdot 3 \cdot 2 \cdot 2 = 216$ configurations, so the search was done in two stages (`scripts/run_irrip_variants.sh` with `scripts/irrip_stage{1,2}.cfg`). *Stage 1* (14 configurations) turned on one change at a time, both on top of SRRIP (an ablation of insertion, promotion and eviction) and on top of I+P, the combination of instruction insertion at RRPV 0 (instruction hits to 0) and data decrement on hit. The all-SRRIP setting reproduces `rrip.l2c_repl` exactly, which validates the knob implementation.

Stage 1: speedup over baseline

Configuration	Trace1	Trace2	Trace3	Mean
SRRIP (all knobs off)	0.9999	1.0022	0.9993	1.0004
Insertion only (instr RRPV 0)	1.0000	1.0130	0.9999	1.0043
Promotion only (data decrement)	1.0008	1.0085	0.9995	1.0029
Eviction only (prefer data)	0.9998	1.0185	0.9993	1.0058
Eviction only (age data faster)	1.0004	1.0088	0.9994	1.0029
Insertion + promotion (I+P)	0.9998	1.0176	0.9999	1.0058
I+P + prefer data	0.9994	1.0229	1.0000	1.0074
I+P + age data faster	0.9993	1.0196	1.0001	1.0063
I+P + both eviction options	0.9983	1.0251	1.0000	1.0077
I+P, instr insert RRPV 1	1.0003	1.0154	0.9999	1.0052
I+P, data insert RRPV 3	0.9968	1.0291	1.0013	1.0089
I+P, data insert bimodal	0.9971	1.0283	1.0008	1.0087
I+P, data not promoted	0.9978	1.0202	1.0000	1.0059
I+P, instr decrement on hit	1.0001	1.0176	1.0001	1.0059



Stage 2 took the knobs that changed the mean speedup by at least 0.05% in stage 1 (instruction insertion, data insertion, both eviction options), each at its two best values, and ran their full factorial ($2^4 = 16$ configurations). Promotion was fixed at instruction-to-0 and data-decrement: the instruction options tied within 0.01%, data-to-0

was clearly worse, and data-unchanged tied with decrement but is unsafe with data inserted at RRPV 3 (see below). Two stage-2 points repeat stage-1 points and reproduced bit-exactly.

Stage 2: speedup over baseline

Instr insert / data insert / prefer data / age data	Trace1	Trace2	Trace3	Mean
instr 1 / data RRPV 3 / prefer on / age on	0.9974	1.0348	1.0020	1.0113
instr 1 / data RRPV 3 / prefer on / age off (chosen)	0.9986	1.0334	1.0020	1.0112
instr 1 / data bimodal / prefer on / age on	0.9980	1.0343	1.0015	1.0111
instr 1 / data RRPV 3 / prefer off / age on	0.9982	1.0337	1.0018	1.0111
instr 1 / data bimodal / prefer on / age off	0.9990	1.0332	1.0014	1.0111
instr 1 / data bimodal / prefer off / age on	0.9986	1.0327	1.0012	1.0107
instr 1 / data RRPV 3 / prefer off / age off	0.9993	1.0313	1.0017	1.0107
instr 1 / data bimodal / prefer off / age off	0.9998	1.0305	1.0012	1.0104
instr 0 / data RRPV 3 / prefer on / age off	0.9961	1.0300	1.0014	1.0091
instr 0 / data RRPV 3 / prefer off / age off	0.9968	1.0291	1.0013	1.0089
instr 0 / data bimodal / prefer on / age on	0.9954	1.0299	1.0011	1.0087
instr 0 / data bimodal / prefer off / age on	0.9963	1.0291	1.0011	1.0087
instr 0 / data bimodal / prefer on / age off	0.9964	1.0290	1.0010	1.0087
instr 0 / data bimodal / prefer off / age off	0.9971	1.0283	1.0008	1.0087
instr 0 / data RRPV 3 / prefer off / age on	0.9955	1.0297	1.0011	1.0087
instr 0 / data RRPV 3 / prefer on / age on	0.9946	1.0300	1.0014	1.0085

Rationale: insertion (modified). Insertion decides how long a line survives before it has proved itself, so it is the most direct lever for prioritising instructions. Code lines are reused across loop iterations and repeated calls, but often at reuse distances larger than the L1I. Inserting data at RRPV 3 makes a new data line the first eviction candidate unless it is reused; this was the largest single improvement in stage 1 (1.0058 to 1.0089), which indicates that most data lines reaching L2 in these traces are not reused before they would be evicted anyway. Instructions are inserted at RRPV 1 rather than 0. This is the one result that only the full factorial revealed: with data inserted at RRPV 2, instruction RRPV 0 was slightly better (1.0058 versus 1.0052), but once data is inserted at RRPV 3, RRPV 1 is better in all eight pairings, by 0.17–0.28%. Data at RRPV 3 already gives code a two-step lead, so RRPV 0 is no longer needed to protect code from data; it only lets code that is fetched once (cold paths, one-time initialisation) push out code that is actually reused. Bimodal data insertion was within 0.04% of RRPV 3 in every stage-2 pairing (and never better) but adds a counter, so plain RRPV 3 was kept.

Rationale: promotion (modified for data only). An instruction hit is strong evidence of a hot code path, so it keeps SRRIP's promote-to-0 rule. The decrement alternative tied (1.0059 versus 1.0058), so the simpler and standard rule was kept. Data uses frequency-based promotion: one hit moves a line one step nearer, so only repeatedly reused data earns the protection code gets on its first hit. Promoting data to 0 on a hit would undo the insertion bias after a single reuse; measured against decrementing, it lowered the mean speedup from 1.0058 to 1.0043. Leaving data unpromoted tied with decrement on its own (1.0059), but combined with insertion at RRPV 3 a reused data line could never leave the eviction position, so decrement is the safe choice.

Rationale: eviction (modified as a tie-breaker only). SRRIP evicts the first line it finds at RRPV 3. Preferring a data line among the RRPV-3 candidates costs nothing extra, because the instruction bit is already stored per line. It only matters when code and data are equally old, and then evicting the data line is consistent with the rest of the policy. On its own it gave the largest gain of any single change on top of SRRIP (1.0004 to 1.0058), and in stage 2 it added about 0.05%. Aging data faster was *not* adopted: on top of the chosen insertion and promotion it moved the mean speedup by at most 0.01% (1.0112 versus 1.0113), but it raised the Trace 1 loss from 0.14% to 0.26% and needs a second aging rule. The aging procedure is therefore left as in SRRIP.

Storage. Per line: 2-bit RRPV (as SRRIP) and 1 instruction bit, which ChampSim's L2C already keeps in `BLOCK`. For 1024 sets $\times$ 8 ways this is 16384 + 8192 = 24576 bits (3 KB), of which only the 8192 instruction bits (1 KB) are on top of SRRIP. No counters or tables are added.

Task 2B: Dynamic I-RRIP

Choice of the I-RRIP component. Task 2A and Task 2B use different I-RRIP settings, because the two tasks are asked different questions. In Task 2A the policy is static: whatever it does to a trace, that trace keeps it. So it was chosen on all three traces, and the setting without faster data aging was kept. Its mean speedup is within noise of the best one (1.0112 versus 1.0113), and it loses less on Trace 1 (0.14% versus 0.26%).

In Task 2B the I-RRIP rules are only one of two options. Set dueling falls back to SRRIP on a workload where they hurt. Trace 1 is such a workload: replacement barely matters there (SRRIP 0.9999 against LRU), and every I-RRIP setting was slightly below the baseline. For Trace 1 the dynamic policy should therefore pick SRRIP, not a gentler I-RRIP. The I-RRIP option only needs to be strong on the workloads where it gets picked. Trace 2 is the case the problem statement describes: a large code footprint, with the highest L2 instruction MPKI under SRRIP (17.98). The I-RRIP option is therefore tuned for Trace 2. Adding faster data aging raises Trace 2's speedup from 1.0334 to 1.0348 and cuts its instruction MPKI from 9.43 to 9.03. The cost is one extra aging rule, used only in sets running I-RRIP.

The fallback is not free. The I-RRIP leader sets (32 of the 1024 sets) always run I-RRIP, so Trace 1 still pays its cost there. Whether the followers really settle on SRRIP for Trace 1 is checked below with the dynamic-policy statistics (final PSEL and the share of follower updates done in each mode).

Task 3: SPP with prefetch-aware Dynamic I-RRIP

Policy (`replacement/dyn_irrip.l2c_repl`, applied in I-RRIP mode only; the RRIP leader sets are unchanged so set dueling still compares against plain RRIP). A prefetched data line is inserted at RRPV 3 (first eviction candidate) instead of 2, and a prefetch request that hits a resident line does not promote it. Instruction lines are unchanged (insert/promote to 0); demand hits on data still promote by one step, so a prefetched line that is actually used moves up. Eviction is unchanged.

Rationale. SPP brings lines into L1D, so the L2 copy of a prefetched line is a duplicate that is only needed if the L1D copy is evicted before use; a speculative line should therefore not displace demand data or instruction lines, which the L2 is supposed to protect. A prefetch hit is not evidence of reuse (the prefetcher re-requests a line it already fetched), so promoting on it would keep stale lines alive. Inserting at the distant position lets a useless prefetch leave quickly, while a demand hit still rescues a useful one. The insertion RRPV was swept (1/2/3); distant insertion is best on mean speedup.

Results (25M warmup + 25M simulation instructions; speedup is geometric mean over the three traces). **IPC**

Config	Trace1	Trace2	Trace3	Mean
Baseline (no pf, LRU)	0.2323	0.2512	0.3821	0.2885
SPP	0.2343	0.2549	0.3872	0.2921
Dynamic IRRIP	0.2323	0.2525	0.3821	0.2889
SPP + Dyn IRRIP (pf-unaware)	0.2342	0.2562	0.3874	0.2926
SPP + Dyn IRRIP (pf-aware)	0.2344	0.2564	0.3875	0.2928
PF insert RRPV 1	0.2342	0.2557	0.3872	0.2923
PF insert RRPV 2	0.2344	0.2561	0.3874	0.2926
PF insert RRPV 3 (final)	0.2344	0.2564	0.3875	0.2928

Speedup over baseline

Config	Trace1	Trace2	Trace3	Mean
Baseline (no pf, LRU)	1.0000	1.0000	1.0000	1.0000
SPP	1.0082	1.0149	1.0135	1.0122
Dynamic IRRIP	0.9997	1.0052	1.0000	1.0017
SPP + Dyn IRRIP (pf-unaware)	1.0078	1.0200	1.0139	1.0139
SPP + Dyn IRRIP (pf-aware)	1.0087	1.0208	1.0143	1.0146
PF insert RRPV 1	1.0079	1.0179	1.0134	1.0131
PF insert RRPV 2	1.0087	1.0197	1.0140	1.0141
PF insert RRPV 3 (final)	1.0087	1.0208	1.0143	1.0146

L1D load MPKI

Config	Trace1	Trace2	Trace3	Mean
Baseline (no pf, LRU)	10.678	13.538	3.403	9.206
SPP	9.653	12.907	3.017	8.526
Dynamic IRRIP	10.678	13.538	3.403	9.207
SPP + Dyn IRRIP (pf-unaware)	9.657	12.906	3.017	8.526
SPP + Dyn IRRIP (pf-aware)	9.661	12.906	3.000	8.523
PF insert RRPV 1	9.657	12.904	3.025	8.529
PF insert RRPV 2	9.656	12.904	3.014	8.525
PF insert RRPV 3 (final)	9.661	12.906	3.000	8.523

L1D prefetch accuracy (%)

Config	Trace1	Trace2	Trace3	Mean
Baseline (no pf, LRU)	–	–	–	–
SPP	52.49	59.53	36.27	49.43
Dynamic IRRIP	–	–	–	–
SPP + Dyn IRRIP (pf-unaware)	52.52	59.68	36.21	49.47
SPP + Dyn IRRIP (pf-aware)	52.08	59.61	36.94	49.54
PF insert RRPV 1	52.03	59.68	35.92	49.21
PF insert RRPV 2	52.35	59.72	36.36	49.48
PF insert RRPV 3 (final)	52.08	59.61	36.94	49.54

L1D load coverage (%)

Config	Trace1	Trace2	Trace3	Mean
Baseline (no pf, LRU)	0.00	0.00	0.00	0.00
SPP	9.60	4.66	11.36	8.54
Dynamic IRRIP	0.00	-0.00	0.00	-0.00
SPP + Dyn IRRIP (pf-unaware)	9.57	4.67	11.36	8.53
SPP + Dyn IRRIP (pf-aware)	9.52	4.67	11.84	8.68
PF insert RRPV 1	9.56	4.68	11.11	8.45
PF insert RRPV 2	9.57	4.68	11.45	8.57
PF insert RRPV 3 (final)	9.52	4.67	11.84	8.68

L2C load MPKI

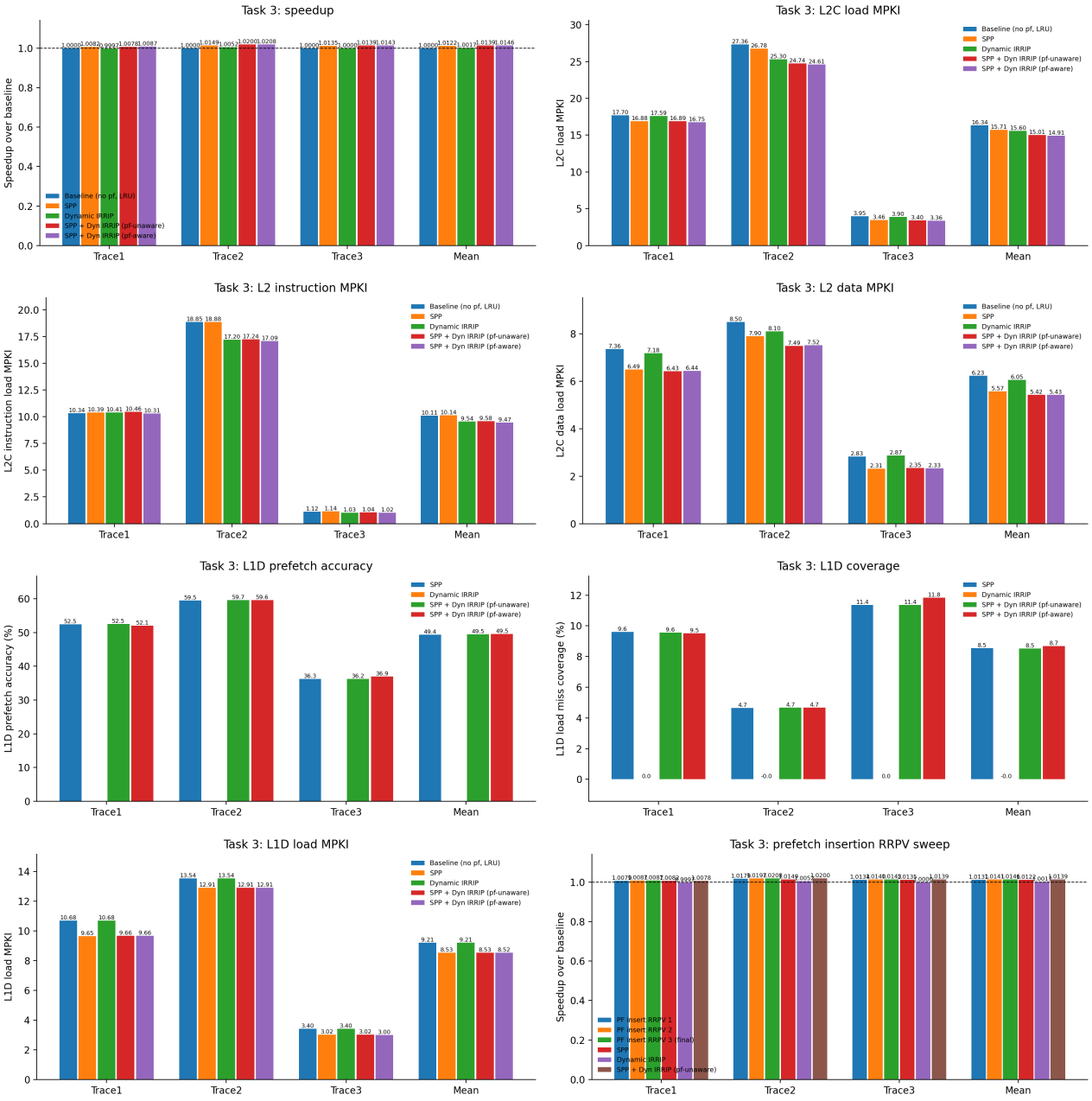
Config	Trace1	Trace2	Trace3	Mean
Baseline (no pf, LRU)	17.696	27.357	3.954	16.336
SPP	16.880	26.784	3.456	15.707
Dynamic IRRIP	17.590	25.298	3.899	15.595
SPP + Dyn IRRIP (pf-unaware)	16.891	24.736	3.396	15.008
SPP + Dyn IRRIP (pf-aware)	16.749	24.609	3.359	14.906
PF insert RRPV 1	16.905	25.093	3.422	15.140
PF insert RRPV 2	16.767	24.779	3.388	14.978
PF insert RRPV 3 (final)	16.749	24.609	3.359	14.906

L2C instruction load MPKI

Config	Trace1	Trace2	Trace3	Mean
Baseline (no pf, LRU)	10.338	18.853	1.124	10.105
SPP	10.389	18.882	1.144	10.138
Dynamic IRRIP	10.408	17.195	1.029	9.544
SPP + Dyn IRRIP (pf-unaware)	10.464	17.244	1.043	9.583
SPP + Dyn IRRIP (pf-aware)	10.306	17.088	1.024	9.472
PF insert RRPV 1	10.475	17.677	1.055	9.736
PF insert RRPV 2	10.335	17.292	1.042	9.556
PF insert RRPV 3 (final)	10.306	17.088	1.024	9.472

L2C data load MPKI

Config	Trace1	Trace2	Trace3	Mean
Baseline (no pf, LRU)	7.358	8.504	2.830	6.231
SPP	6.491	7.903	2.312	5.569
Dynamic IRRIP	7.181	8.102	2.869	6.051
SPP + Dyn IRRIP (pf-unaware)	6.427	7.493	2.353	5.424
SPP + Dyn IRRIP (pf-aware)	6.443	7.522	2.335	5.433
PF insert RRPV 1	6.431	7.417	2.367	5.405
PF insert RRPV 2	6.431	7.487	2.346	5.422
PF insert RRPV 3 (final)	6.443	7.522	2.335	5.433



Findings. SPP + prefetch-aware Dyn I-RRIP reaches mean speedup 1.0146, against 1.0122 for SPP alone, 1.0017 for Dynamic I-RRIP alone and 1.0139 for SPP with the prefetch-unaware Dynamic I-RRIP; its IPC is highest on all three traces (0.2344/0.2564/0.3875), so it outperforms both SPP and Dynamic I-RRIP in IPC. The margins are small: on Trace 1 the gain over SPP is ≈0.0001 IPC, and the gain over the prefetch-unaware policy is 0.0007 in mean speedup, within noise for a single 25M-instruction window. The effect is clearest at L2: mean L2 load MPKI falls from 15.01 (unaware) to 14.91, and L2 instruction load MPKI from 9.58 to 9.47, because prefetch fills no longer displace instruction lines. L2 data load MPKI is slightly higher than unaware (5.43 vs 5.42), and prefetch accuracy and coverage at L1D are essentially unchanged (49.5% / 8.7%), since the change affects L2 residency, not what is prefetched.

Limitations

One 25M-instruction window per trace (the simulator is deterministic, but differences of ∼0.1% may not generalise). ChampSim's warmup skips front-end stalls, so the measured region starts with caches and tables relatively cold;

this affects all configurations equally. The accuracy counter does not track prefetches that fill L2.